Chapter 15: Binary Search and Assorted Algorithmic Techniques

EECS 281 notes, condensed. Covers 15.1 to 15.4.

Legend

- **LOWER PRIORITY** = Sections 15.3 and 15.4. Per course directions, the additional algorithms after section 15.2 are **not as important** as the earlier material. You will most likely not be asked to implement things like Moore's voting algorithm on an exam. Included in full for completeness, marked with a red left bar.
- **Sections 15.1 and 15.2 are the core, higher-priority content of this chapter.**
- Boxes: **blue** = definition, **green** = worked example, **yellow** = remark / detail.

15.1 Ordered and Sorted Containers

Ordered container: maintains its current ordering of elements regardless of how the container is modified. Elements keep their **relative positions** unless they are removed.

- If A comes before B, and you insert C, then A is still before B.
- If C is then removed, A is still before B.
- As long as A and B are both in the container, A stays before B, no matter what else is inserted or removed.
- In the STL, ordered containers typically support an **insertion method that takes an iterator** defining where the new element goes.

Sorted container: a sequential container whose elements are always in some **predefined order**.

- You **cannot** insert wherever you want. A new element's position must follow the predefined ordering.
- Example: a sorted container holds 1 and 3. Inserting 2 must place it between 1 and 3 (ascending order).

Examples seen so far

- **Linked lists, arrays, deques:** ordered on their own (inserting or deleting does not change the relative order of the rest). They are **not** sorted by default, since nothing dictates value order.
- These same containers can be used as the **underlying container** of other structures that may be unordered or sorted.
- **Unordered sequence container priority queue** (section 10.2.2) is **unordered**, because popping changes the relative order of other elements:
  `[16, 22, 15, 11, 19]  .pop()→ overwrite 22 with 19 → [16, 19, 15, 11, 19] → pop back → [16, 19, 15, 11]`
- **Sorted sequence container priority queue** (section 10.2.3) uses an underlying array required to follow a predefined order, so it is a **sorted container**:
  `[13,17,24,33,42] .push(19) → [13,17,19,24,33,42]  then .pop() → [13,17,19,24,33]`
- **Set implementation** from section 13.1.2 uses a **sorted array** as its underlying structure (that invariant is what makes set operations efficient), so it is also a sorted container.

Choosing the underlying container

- Arrays, linked lists, and deques are all reasonable choices for storing the underlying data.
- The right choice depends on the application and which operations are called frequently. For example, if you need random access, a list is not the way to go.

Worst-case complexities: ORDERED container

Operation	Array	Linked List	Deque
add_value(val) insert value anywhere	Theta(1)	Theta(1)	Theta(1)
remove(val) must find it first	Theta(n)	Theta(n)	Theta(n)
remove(iterator) iterator given	Theta(n)	Theta(n) if singly-linked Theta(1) if doubly-linked	Theta(n)
find(value)	Theta(n)	Theta(n)	Theta(n)
iterator::operator*() dereference	Theta(1)	Theta(1)	Theta(1)
operator[](index)	Theta(1)	Theta(n)	Theta(1)
insert_after(iterator, val)	Theta(n)	Theta(1)	Theta(n)
insert_before(iterator, val)	Theta(n)	Theta(n) if singly-linked Theta(1) if doubly-linked	Theta(n)

Worst-case complexities: SORTED container

Operation	Array	Linked List	Deque
add_value(val)	Theta(n)	Theta(n)	Theta(n)
remove(val) must find it first	Theta(n)	Theta(n)	Theta(n)
remove(iterator)	Theta(n)	Theta(n) if singly-linked Theta(1) if doubly-linked	Theta(n)
find(value)	**Theta(log(n))**	Theta(n)	**Theta(log(n))**
iterator::operator*()	Theta(1)	Theta(1)	Theta(1)
operator[](index)	Theta(1)	Theta(n)	Theta(1)
insert_after(iterator, val)	N/A	N/A	N/A
insert_before(iterator, val)	N/A	N/A	N/A

Key differences between the two tables

- **add_value becomes Theta(n)** in a sorted container, because the new element must go in a position that keeps sorted order.
  - Linear for a **list**, since you cannot use binary search on it.
  - Linear for **vectors and deques**, since elements may need to be shifted after the insertion.
- **find improves to Theta(log(n))** for random-access containers (array, deque), because binary search becomes available. It stays **Theta(n)** for a list, which has no random access.
- **insert_before and insert_after are no longer supported**, because you do not get to choose where values go in a sorted container.

Example 15.1: Is the underlying data vector of a binary heap sorted? Is it ordered?

Neither.

Not sorted: the heap invariant only requires that no parent has lower priority than any of its descendants. That leaves multiple valid vector representations of the same elements.

```
min-heap A: root 1, children 3 and 2, then 6, 5, 4  → vector [1, 2, 3, 4, 5, 6]
min-heap B: root 1, children 2 and 4, then 3, 6, 5  → vector [1, 3, 2, 6, 5, 4]
```

Both are valid min-heaps, so the data does not follow a single predefined order.

Not ordered: insertion can change the relative order of existing elements. Take the max-heap [8, 2, 4] and insert 9:

```
[8, 2, 4]  insert 9 at the back→ [8, 2, 4, 9]
fix up: 9 swaps with 2 → [8, 9, 4, 2]
fix up: 9 swaps with 8 → [9, 8, 4, 2]
```

Before the insert, 2 came before 4. After the insert, 2 comes after 4. The relative order changed, so the vector is not an ordered container.

15.2 Binary Search

15.2.1 Binary Search Implementation

Binary search: finds a value in **logarithmic time**, but only if the underlying data is **sorted** and supports random access.

- Look at the **middle** element and compare it to the target.
  - Middle is **larger** than the target: the target must be to the **left**.
  - Middle is **smaller** than the target: the target must be to the **right**.
- Repeat, **eliminating half the search space each time**, until the target is found or the space is empty.

Example: search for 21 in [2, 3, 5, 7, 13, 21, 25, 31, 42]

```
indices  0  1  2  3   4   5   6   7   8
Middle is 13. 21 > 13, so eliminate everything left of 13.
Remaining [21, 25, 31, 42], middle is 25 (integer division picks the left of two middles).
21 < 25, so eliminate everything right of 25. Remaining [21].
21 equals the target. Found at index 5.
```

```
int32_t binary_search(const std::vector<int32_t>& vec, int32_t target,
                      int32_t left, int32_t right) {
  while (right > left) {
    int mid = left + (right - left) / 2;
    if (target == vec[mid]) {
      return mid;
    } // if
    if (target < vec[mid]) {
      right = mid;
    } // if
    else {
      left = mid + 1;
    } // else
  } // while
  return -1;   // target not found
} // binary_search()
```

- `[left, right)` is the range being searched. To search everything, start with `left = 0` and `right = size`.
- The while loop runs while elements remain. Half the search space is eliminated per iteration, so the complexity is **Theta(log(n))**.

Note: use `left + (right - left) / 2`, not `(left + right) / 2`. Adding left and right could **overflow** if the indices are very large.

Optimization: check < and > before ==

- The `==` check rarely returns true, since the target is usually less than or greater than the element you check.
- Testing `<` and `>` first saves comparison checks per iteration.

```
int32_t binary_search(const std::vector<int32_t>& vec, int32_t target,
                      int32_t left, int32_t right) {
  while (right > left) {
    int32_t mid = left + (right - left) / 2;
    if (target < vec[mid]) {
      right = mid;
    } // if
    else if (target > vec[mid]) {
      left = mid + 1;
    } // else if
    else {
      return mid;
    } // else
  } // while
  return -1;   // target not found
} // binary_search()
```

Recursive binary search

- Recurse right if the middle is smaller than the target, recurse left if the middle is larger.

```
int32_t binary_search(const std::vector<int32_t>& vec, int32_t target,
                      int32_t left, int32_t right) {
  if (left > right) {          // base case, failed to find value
    return -1;
  } // if
  int32_t mid = left + (right - left) / 2;
  if (vec[mid] < target) {
    return binary_search(vec, target, mid + 1, right);
  } // if
  if (vec[mid] > target) {
    return binary_search(vec, target, left, mid - 1);
  } // if
  return mid;
} // binary_search()
```

Lower bound

Lower bound: instead of checking whether the element exists, it returns the **index the element would be found at if it did exist**. Useful for finding the correct insertion position in a sorted container.

- Since there is no equality check, only **two comparisons per loop** are made instead of three.

```
int32_t lower_bound(const std::vector<int32_t>& vec, int32_t target,
                    int32_t left, int32_t right) {
  while (right > left) {
    int32_t mid = left + (right - left) / 2;
    if (vec[mid] < target) {
      left = mid + 1;
    } // if
    else {
      right = mid;
    } // else
  } // while
  return left;
} // lower_bound()
```

Example: lower bound of 20 in [2, 3, 5, 7, 13, 21, 25, 31, 42]
`left = index 0, right = index 8, mid points at 13. 20 > 13, so left moves to the position of 21.`
`mid is now 25, which is larger than 20, so right moves to the position of 25.`
`mid is now 21, still larger than 20, so right moves to the position of 21.`
`left and right now refer to the same element, the loop exits, and` **`left (index 5) is returned`**.
20 is not in the array, but index 5 is where it would go if it existed.

15.2.2 Solving Problems Using Binary Search

Binary search is a classic interview topic. The common pattern: find a property that **splits the array into two segments**, so you can tell which half the answer is in and discard the other half.

Example 15.2: array is increasing then decreasing. Find the maximum. Given [1, 3, 50, 10, 9, 7, 6], return 50.

- The maximum splits the array: everything to its left is **ascending**, everything to its right is **descending**. That lets us tell which side of the max we are on.
- Look at the middle element and compare it with its two neighbors:
  - Larger than **both** neighbors: it is the maximum, return it.
  - Greater than its left neighbor and smaller than its right: we are on the **ascending** part, so the max is to the **right**.
  - Smaller than its left neighbor and greater than its right: we are on the **descending** part, so the max is to the **left**.

```
int32_t find_max_value(const std::vector<int32_t>& vec, int left, int right) {
  int32_t start = left, end = right - 1;
  while (start < end) {
    int32_t mid = start + (end - start) / 2;
    // target must be to the right of mid
    if (vec[mid] < vec[mid + 1] && vec[mid] > vec[mid - 1]) {
      start = mid + 1;
    } // if
    else {   // target must be to the left of mid
      end = mid;
    } // else
  } // while
  return vec[start];
} // find_max_value()
```

Time Theta(log(n)), auxiliary space Theta(1).

Example 15.3: sorted array rotated at some position. Return the minimum. Given [6, 7, 1, 2, 3, 4, 5], return 1.

- The minimum splits the array again: everything **left** of the min is **larger than the last element**, and everything from the min onward is **less than or equal to the last element**.
- Compare the middle element with the **last** element of the current range:
  - Middle is smaller than the last: everything right of middle can be eliminated.
  - Middle is larger than the last: everything left of middle can be eliminated.

```
int32_t find_min_rotated(const std::vector<int32_t>& vec, int32_t left, int32_t right) {
  int32_t start = left, end = right - 1;
  while (start < end) {
    int32_t mid = start + (end - start) / 2;
    // target must be to the right of mid
    if (vec[mid] > vec[end]) {
      start = mid + 1;
    } // if
    else {   // target must be to the left of mid
      end = mid;
    } // else
  } // while
  return vec[start];
} // find_min_rotated()
```

Time Theta(log(n)), auxiliary space Theta(1).

Example 15.4: sorted array where every number occurs twice except one. Return the single number. Given [1, 1, 2, 2, 3, 4, 4], return 3.

- Splitting property (subtle): for a pair occurring **before** the single element, the first element of the pair sits at an **even** index (the first 1 is at index 0, the first 2 is at index 2). For a pair occurring **after** the single element, the first element of the pair sits at an **odd** index (the first 4 is at index 5).
- Look at the middle index:
  - If mid is **even**, check whether `vec[mid] == vec[mid + 1]`.
  - If mid is **odd**, check whether `vec[mid] == vec[mid - 1]`.
  - If true, the single element is to the **right**. Otherwise it is to the **left**.

```
int32_t find_single_element(const std::vector<int32_t>& vec, int32_t left, int32_t right) {
  int32_t start = left, end = right - 1;
  while (start < end) {
    int mid = start + (end - start) / 2;
    // target must be to the right of mid
    if ((mid % 2 == 0 /* even */ && vec[mid] == vec[mid + 1]) ||
        (mid % 2 == 1 /* odd  */ && vec[mid] == vec[mid - 1])) {
      start = mid + 1;
    } // if
    else {   // target must be to the left of mid
      end = mid;
    } // else
  } // while
  return vec[start];
} // find_single_element()
```

Time Theta(log(n)), auxiliary space Theta(1).

15.3 Algorithmic Techniques for Linear Data Structures LOWER PRIORITY

Per course directions: the additional algorithms in sections 15.3 and beyond are **not as important** as the earlier material. You will most likely not be asked to implement Moore's voting algorithm or similar on an exam. These techniques may not be covered explicitly in class, but they are still worth being familiar with because they are useful

patterns for approaching problems on the data structures covered so far.
Everything below this banner until the end of the chapter is marked with a **red left bar**.

15.3.1 Two Pointer Technique

- Start with **two pointers (or indices)**: one at the first element, one at the last element.
- Move them **toward each other**, using the values they reference at each step to solve the problem.
- Because each pointer only moves inward, every value is visited once, which gives a **Theta(n)** pass.

Example 15.5: find a pair in a sorted ascending array that sums to a target. Given [3, 4, 7, 9, 10] and target 11, return [4, 7]. Return [-1, -1] if no pair works.

- Naive: check every pair, Theta(n^2).
- Because the array is sorted, the sum of the two pointed-at values tells you which way to move:
  - Sum **too large**: decrement right.
  - Sum **too small**: increment left.
- If left and right ever meet, no pair sums to the target.

```
[3, 4, 7, 9, 10], L at 3, R at 10 → sum 13, too large, --R
L at 3, R at 9 → sum 12, too large, --R
L at 3, R at 7 → sum 10, too small, ++L
L at 4, R at 7 → sum 11, equals target. Return [4, 7].
```

```
std::pair<int32_t, int32_t> target_sum(const std::vector<int32_t>& vec, int32_t target) {
  size_t left = 0, right = vec.size() - 1;
  while (left < right) {
    if (vec[left] + vec[right] < target) {
      ++left;
    } // if
    else if (vec[left] + vec[right] > target) {
      --right;
    } // else if
    else {
      return {vec[left], vec[right]};
    } // else
  } // while
  return {-1, -1};   // no solution
} // target_sum()
```

Time Theta(n), auxiliary space Theta(1).

Example 15.6: maximum water held between vertical bars. Given heights [1, 4, 9, 5, 8, 10, 7, 6, 3], each bar 1 unit apart, the answer is 30.

- Water held by two bars = `min(height) * (distance between them)`.
- Start with the leftmost and rightmost bars. Then move the pointer at the **shorter** bar inward, since only a taller bar can possibly improve the result.
- Keep moving that pointer until you find a bar **taller** than any seen so far on that side.

```
bars 1 and 9 (indices 1 and 9): 1 * (9 - 1) = 8  → best = 8
move left (shorter). bars 4 and 3: 3 * (9 - 2) = 21  → best = 21
move right (shorter). 4 * (8 - 2) = 24  → best = 24
move left (shorter). 6 * (8 - 3) = 30  → best = 30
move right. 7 * (7 - 3) = 28, not better, ignore
move right. 9 * (6 - 3) = 27, not better, ignore
move left until a taller bar is found. This skips bars 4 and 5 and makes the pointers meet.
Algorithm complete, answer 30.
```

```
int32_t max_water(const std::vector<int32_t>& bars) {
  size_t left = 0, right = bars.size() - 1;
  int32_t best = 0;
  while (left < right) {
    int32_t min_height = std::min(bars[left], bars[right]);
    best = std::max(best, min_height * (right - left));
    while (bars[left] <= min_height && left < right) {
      ++left;
    } // while
    while (bars[right] <= min_height && left < right) {
      --right;
    } // while
  } // while
  return best;
} // max_water()
```

Time Theta(n), auxiliary space Theta(1).

15.3.2 Sliding Window Technique

- Considers **individual subsets (windows)** of the data and **expands or shrinks** the window based on the problem's conditions, giving the effect of sliding a window through the data.
- **Rule of thumb:** useful when you are given an ordered and iterable container (vector, string) and asked about a **subrange** of it, for example the longest or shortest subrange satisfying a condition.

Caveat: this is only a rule of thumb. Not every problem with this shape can be solved by a sliding window. Some need dynamic programming or divide-and-conquer instead. It is still a good pattern to keep in your toolbox.

Example 15.7: length of the longest substring with no repeating characters. Given "eecsiseecs", return 4 (the substring "ecsi"). Assume only lowercase a-z.

- Start at the left edge and expand the window one character at a time with the right boundary.
- Track which letters are currently in the window, and the best window length seen.
- If the right boundary adds a **duplicate**, advance the **left** boundary until the duplicate is gone.

```
add e → window {e}, best = 1
add second e → duplicate, advance left by one → window {e}, best = 1
add c → {e, c}, best = 2
add s → {e, c, s}, best = 3
add i → {e, c, s, i}, best = 4
add s → duplicate, advance left until only one s: {c,s,i} then {s,i} then {i}, then add s → {i, s}, best still 4
add e → {i, s, e}
add second e → duplicate, advance left until the other e is gone → {e}
add c → {e, c}; add s → {e, c, s}
right boundary reaches the end. Answer 4.
```

```cpp
int32_t length_of_longest_substring(const std::string& str) {
  std::vector<int8_t> count(26);   // which letters are in the window
  size_t left_idx = 0, right_idx = 0;
  int32_t best = 0;

  while (right_idx < str.length()) {
    ++count[str[right_idx] - 'a'];             // add char at right_idx
    while (count[str[right_idx] - 'a'] > 1) {   // runs only if there is a duplicate
      --count[str[left_idx] - 'a'];            // remove char at left_idx
      ++left_idx;                              // slide left forward
    } // while
    int32_t current_length = right_idx - left_idx + 1;
    best = std::max(best, current_length);
    ++right_idx;
  } // while

  return best;
} // length_of_longest_substring()
```

Time complexity, why it is not Theta(n^2): the nested loop looks quadratic, but the two loops are **dependent**. The inner loop only runs while the count of the character at `right_idx` exceeds one, and that count is bounded by how many times `right_idx` was incremented in the outer loop. The inner loop can run at most once per outer iteration, and the outer loop runs n times, so the combined complexity is **Theta(n)**.
Auxiliary space Theta(1): only a few variables and a fixed vector of size 26, none of which depend on the string length.

Example 15.8: shortest substring of str containing every character of target (matching frequencies). Given str = "transcendences" and target = "eecs", return "ences" (2 e's, 1 c, 1 s). Return "" if no such substring exists. Assume lowercase a-z and a unique solution.

- The **right** index expands the window, the **left** index contracts it.
- Define a **viable window** as one containing all target characters with matching frequencies.
- Expand right until the window is viable. Record it as a possible solution.
- Then move left forward while the window stays viable, recording each smaller viable window.
- Once the window is no longer viable, go back to expanding right. Stop when right runs off the end.

```
expand right through t,r,a,n,s,c,e,n,d,e → first viable window "transcende"
contract left: "ranscende", "anscende", "nscende", "scende" (all still viable)
contracting once more breaks viability, so best so far = "scende"
expand right through n, c, e → viable again
contract left repeatedly, eventually reaching "ences" (viable and shorter) → best = "ences"
one more contraction breaks viability; expanding right runs off the end
Answer: "ences"
```

```cpp
std::string min_window_substring(const std::string& str, const std::string& target) {
  if (str.length() == 0 || target.length() == 0) {
    return "";   // no solution if either string is empty
  } // if

  // character frequency in window and in target ('a' = 0, ..., 'z' = 25)
  std::vector<int32_t> count_in_window(26);
  std::vector<int32_t> count_in_target(26);
  for (size_t i = 0; i < target.length(); ++i) {
    ++count_in_target[target[i] - 'a'];
  } // for i

  // number of characters matched in the window
  // if this equals target.length(), the window is viable
  size_t characters_matched = 0;
  size_t left_idx = 0, right_idx = 0;
  std::string best;

  while (right_idx < str.length()) {
    char right_idx_char = str[right_idx];
    ++count_in_window[right_idx_char - 'a'];
    // target char added and not over frequency, so count it
    if (count_in_window[right_idx_char - 'a'] <= count_in_target[right_idx_char - 'a']) {
      ++characters_matched;
    } // if
    // while viable, contract the window from the left
    while (left_idx <= right_idx && characters_matched == target.length()) {
      size_t window_length = right_idx - left_idx + 1;
      if (best.empty() || window_length < best.length()) {
        best = str.substr(left_idx, window_length);
      } // if
      char left_idx_char = str[left_idx];
      --count_in_window[left_idx_char - 'a'];
      // if a needed char was removed, window is no longer viable
      if (count_in_target[left_idx_char - 'a'] > 0
          && count_in_window[left_idx_char - 'a'] < count_in_target[left_idx_char - 'a']) {
        --characters_matched;
      } // if
      ++left_idx;
    } // while
    ++right_idx;   // expand the window
  } // while
  return best;
} // min_window_substring()
```

Time Theta(s + t), where s and t are the two string lengths: the sliding window is linear in s, and the target is scanned once.

15.3.3 Boyer-Moore Majority Voting Algorithm

- **Problem:** given an array of size n where one element repeats more than n/2 times (a **majority element**), identify it in **Theta(n)** time.
- Keep a **counter** and a **candidate**. Steps:
  1. Initialize the candidate to the first element and the counter to 1.

2. Continue traversing. If an element equals the candidate, **increment** the counter.
3. If an element does not equal the candidate, **decrement** the counter.
4. If the counter ever hits **0**, set the candidate to the element currently being visited and reset the counter to 1.
5. Repeat until the array is processed. The remaining candidate is guaranteed to be the majority element **if one exists**.
6. If a majority element is not guaranteed to exist, **traverse again** to verify the candidate actually appears more than n/2 times.

- **Why it works:** each element "votes" for itself as candidate and opposing votes cancel out. A true majority element has enough votes to survive the cancellation.

Example: [3, 1, 3, 3, 1]

```
visit 3 → matches candidate 3, counter = 1
visit 1 → no match, counter = 0
counter is 0, so candidate becomes 1, counter = 1
visit 3 → no match, counter = 0
counter is 0, so candidate becomes 3, counter = 1
visit 3 → match, counter = 2
visit 1 → no match, counter = 1
Final candidate = 3. Verify: 3 appears 3 times > floor(5/2) = 2. Confirmed.
```

Why the verification pass is necessary: take [4, 4, 3, 3, 1], which has **no** majority element. Following the same rules ends with candidate = 1, counter = 1. But 1 is clearly not a majority element. Without the second pass you would report a wrong answer.

```
int32_t majority_element(const std::vector<int32_t>& vec) {
  int32_t counter = 0, candidate;
  for (int32_t val : vec) {
    if (counter == 0) {
      candidate = val;
    } // if
    counter += (val == candidate) ? 1 : -1;
  } // for val
  int32_t threshold = vec.size() / 2;
  counter = 0;
  for (int32_t val : vec) {          // verification pass
    if (val == candidate) {
      ++counter;
    } // if
  } // for val
  if (counter > threshold) {
    return candidate;
  } // if
  return -1;
} // majority_element()

int main() {
  std::vector<int32_t> vec = {3, 1, 3, 3, 1};
  int32_t majority = majority_element(vec);
  if (majority != -1) {
    std::cout << "The majority element is " << majority << '\n';
  } // if
  else {
    std::cout << "There is no majority element.\n";
  } // else
} // main()
```

Footnote: using -1 to mean "no solution" is not ideal, since -1 could itself be the majority element. `std::optional<>` would be a better choice.

15.3.4 Monotonic Stacks and Queues

Monotonic stack / queue: a stack or queue whose elements are kept in strictly ascending or descending order. The difference from a normal stack or queue is in **push**: before pushing, check whether the new element breaks the monotonic condition. If it does, **pop out every element that violates it** first, then push.

- Useful for: finding the next larger or smaller element in a list, or finding the max or min inside a sliding window.
- **Recurring complexity insight:** these solutions look quadratic because of a nested loop, but each element is pushed and popped at most once, so the total work is **Theta(n)**.

Example 15.9: daily temperatures, days to wait for a warmer temperature. Given [25, 32, 16, 22, 21, 20, 21, 23, 33], return [1, 7, 1, 4, 3, 1, 1, 1, 0]. Store 0 if no warmer day exists.

- Brute force: for each day, scan all later days, Theta(n^2).
- **Key insight:** if several consecutive days have **descending** temperatures, they all share the same "warmer day" once one appears. So **defer** the answer for those days and backfill them all at once when a warmer temperature shows up.
- This needs LIFO access to the deferred days, and the deferred days must be in descending temperature order, which is exactly a **descending monotonic stack**.
- Store **indices**, not temperatures, so the wait time is just a subtraction (and the temperature is one lookup away).

```
day 0 (25): stack empty, push 0 → stack [0]
day 1 (32): 32 > 25, pop 0, solution[0] = 1 - 0 = 1. Push 1 → stack [1]
day 2 (16): 16 < 32, push 2 → stack [1, 2]
day 3 (22): 22 > 16, pop 2, solution[2] = 3 - 2 = 1. 22 < 32, push 3 → stack [1, 3]
day 4 (21): push 4 → stack [1, 3, 4]
day 5 (20): push 5 → stack [1, 3, 4, 5]
day 6 (21): 21 > 20, pop 5, solution[5] = 1. 21 is not warmer than 21, push 6 → [1, 3, 4, 6]
day 7 (23): pop 6, solution[6] = 1. pop 4, solution[4] = 7 - 4 = 3. pop 3, solution[3] = 7 - 3 = 4.
    23 < 32, push 7 → stack [1, 7]
day 8 (33): pop 7, solution[7] = 1. pop 1, solution[1] = 8 - 1 = 7. Push 8 → stack [8]
Done iterating. Anything left in the stack has no warmer day, so solution[8] = 0.
Final: [1, 7, 1, 4, 3, 1, 1, 1, 0]
```

```
std::vector<int32_t> warmer_temperatures(const std::vector<int32_t>& temps) {
  std::vector<int32_t> solution(temps.size());
  std::stack<int32_t> waiting;
  for (size_t curr_day = 0; curr_day < temps.size(); ++curr_day) {
    int32_t curr_temp = temps[curr_day];
    // pop until the current temp is not warmer than the day on top
    while (!waiting.empty() && temps[waiting.top()] < curr_temp) {
      int32_t prev_day = waiting.top();
      solution[prev_day] = curr_day - prev_day;
      waiting.pop();
    } // while
    waiting.push(curr_day);
  } // for curr_day
  return solution;
} // warmer_temperatures()
```

Time Theta(n): each element is pushed once, so the stack can only be popped n times total. Each inner-loop iteration performs exactly one pop, so the inner loop cannot exceed n pops overall.

Example 15.10: smallest integer obtainable after removing exactly k digits. Given num = "453178629" and k = 4, return "17629".

- Goal: remove the **largest digits in the most significant positions**. Iterate left to right (decreasing significance) so higher positions get priority.
- The container must track both **significance position** (order digits were seen) and **which digit to remove** (sorted order). An **ascending monotonic stack** gives both.
- The largest kept digit is on top, so it can be popped when a smaller, more worthwhile digit arrives.
- Restrict the stack to **exactly k pops**. After the k-th pop, all remaining digits are kept.
- If you reach the end without using all k pops, pop the excess off the top. That is safe because the stack is sorted, so the larger digits go first.

```
digit 4: stack empty, push → [4], pops left 4
digit 5: not better than 4, push → [4, 5], pops left 4
digit 3: better than 5, pop 5 → [4], pops left 3. Still better than 4, pop 4 → [], pops left 2. Push 3 → [3]
digit 1: better than 3, pop 3 → [], pops left 1. Push 1 → [1]
digit 7: not better than 1, push → [1, 7], pops left 1
digit 8: not better than 7, push → [1, 7, 8], pops left 1
digit 6: better than 8, pop 8 → [1, 7], pops left 0
out of pops, so push the rest: 6, 2, 9 → [1, 7, 6, 2, 9]
Answer "17629"
```

```
std::string min_remove_k_digits(const std::string& num, int32_t k) {
  int32_t counter = k;
  // vector used for iterator support, behaves like a stack (.back() == .top())
  std::vector<char> monostack;
  for (char curr_digit : num) {
    while (counter > 0 && !monostack.empty() && monostack.back() > curr_digit) {
      monostack.pop_back();
      counter -= 1;
    } // while
    // curr_digit != '0' check prevents leading zeros in the solution
    if (!monostack.empty() || curr_digit != '0') {
      monostack.push_back(curr_digit);
    } // if
  } // for curr_digit
  // if pops remain, keep popping; safe because the stack is sorted
  while (!monostack.empty() && counter-- != 0) {
    monostack.pop_back();
  } // while
  return monostack.empty() ? "0" : std::string{monostack.begin(), monostack.end()};
} // min_remove_k_digits()
```

Alternative: `std::string` supports `.push_back()` and `.pop_back()`, so a string can be used as the stack instead of a vector. Then the solution can be returned directly with no conversion:

```
std::string min_remove_k_digits(const std::string& num, int32_t k) {
  int32_t counter = k;
  std::string monostack;          // use a string to emulate a stack
  for (char curr_digit : num) {
    while (counter > 0 && !monostack.empty() && monostack.back() > curr_digit) {
      monostack.pop_back();
      counter -= 1;
    } // while
    if (!monostack.empty() || curr_digit != '0') {
      monostack.push_back(curr_digit);
    } // if
  } // for curr_digit
  while (!monostack.empty() && counter-- != 0) {
    monostack.pop_back();
  } // while
  return monostack.empty() ? "0" : monostack;
} // min_remove_k_digits()
```

Time Theta(n) by the same push-once, pop-once argument.

Example 15.11: maximum value of each sliding window of size k. Given nums = [1, 9, 8, 6, 7, 2, 4, 5] and k = 3, return [9, 9, 8, 7, 7, 5].

Window position	Max
[1, 9, 8], 6, 7, 2, 4, 5	9
1, [9, 8, 6], 7, 2, 4, 5	9
1, 9, [8, 6, 7], 2, 4, 5	8
1, 9, 8, [6, 7, 2], 4, 5	7
1, 9, 8, 6, [7, 2, 4], 5	7
1, 9, 8, 6, 7, [2, 4, 5]	5

- **Naive:** check every window, Theta(nk), since there are n-k+1 windows each of size k.
- **Priority queue approach:** store pairs of (value, index) so you can find the largest value at a valid index. Works, but pushing and popping are both Theta(log(n)).

```
std::vector<int32_t> max_sliding_window(const std::vector<int32_t>& nums, int32_t k) {
  std::priority_queue<std::pair<int32_t, int32_t>> pq;   // pair<value, index>
  std::vector<int32_t> solution;
  solution.reserve(nums.size() - k + 1);
  for (size_t i = 0; i < k; ++i) {          // first window
    pq.emplace(nums[i], i);
  } // for i
  solution.push_back(pq.top().first);
  for (size_t j = k; j < nums.size(); ++j) {
    pq.emplace(nums[j], j);
    while (!(pq.top().second > j - k)) {    // drop elements outside the window
      pq.pop();
    } // while
    solution.push_back(pq.top().first);
  } // for j
  return solution;
} // max_sliding_window()
```

- **Better: monotonic queue.** We want constant-time insert, constant-time removal of old values, and constant-time access to the window maximum.
- **Key insight:** when pushing a new element, any value in the window **smaller** than it can never be a future window maximum, because those values leave the window before the new element does. So pop them all out. That keeps the queue in **descending** order.
- The window max is then always the value at the **front** of the monotonic queue, readable in Theta(1).
- Store **indices** so expired values can be detected. Note this container must support popping from the **back** as well as the front, so a `std::deque<>` is used rather than a plain queue.

```
idx 0 (1): queue empty, push → [0]
idx 1 (9): back value 1 < 9, pop it, push 1 → [1]
idx 2 (8): back value 9 > 8, just push → [1, 2]. First window ready, max = nums[1] = 9
idx 3 (6): index 0 already gone. 6 is smaller than 9 and 8, push → [1, 2, 3]. Max = nums[1] = 9
idx 4 (7): remove index 1 (expired). Pop index 3 (6 < 7), push 4 → [2, 4]. Max = nums[2] = 8
idx 5 (2): remove index 2 (expired). 2 < 7, push 5 → [4, 5]. Max = nums[4] = 7
idx 6 (4): index 3 already gone. Pop index 5 (2 < 4), push 6 → [4, 6]. Max = nums[4] = 7
idx 7 (5): remove index 4 (expired). Pop index 6 (4 < 5), push 7 → [7]. Max = nums[7] = 5
Solution [9, 9, 8, 7, 7, 5]
```

```
std::vector<int32_t> max_sliding_window(const std::vector<int32_t>& nums, int32_t k) {
  if (nums.empty()) {
    return nums;
  } // if
  std::deque<int32_t> monoqueue;
  std::vector<int32_t> solution;
  solution.reserve(nums.size() - k + 1);
  for (int32_t curr_idx = 0; curr_idx < nums.size(); ++curr_idx) {
    // remove old values no longer in the sliding window
    while (!monoqueue.empty() && monoqueue.front() < curr_idx - k + 1) {
      monoqueue.pop_front();
    } // while
    // remove values smaller than the newest value, they can never be a max
    while (!monoqueue.empty() && nums[monoqueue.back()] < nums[curr_idx]) {
      monoqueue.pop_back();
    } // while
    monoqueue.push_back(curr_idx);
    if (curr_idx - k + 1 >= 0) {                    // write window maximum
      solution.push_back(nums[monoqueue.front()]);
    } // if
  } // for curr_idx
  return solution;
} // max_sliding_window()
```

Time Theta(n): each element is pushed and popped from the monotonic queue at most once.

15.4 Median Finding Algorithms LOWER PRIORITY

- **Simple approach:** sort the container and take the value at index n/2 (odd size) or the average of the values at indices n/2 and floor(n/2) - 1 (even size).

```
double find_median(const std::vector<double>& vec) {
  std::vector<double> sorted{vec};
  size_t sz = sorted.size();
  if (sz == 0) {
    std::cerr << "Vector cannot be empty!" << std::endl;
    throw std::invalid_argument("Cannot find median of an empty vector");
  } // if
  std::sort(sorted.begin(), sorted.end());
  if (sorted.size() % 2 == 1) {           // odd
    return sorted[sz / 2];
  } // if
  return 0.5 * (sorted[sz / 2 - 1] + sorted[sz / 2]);   // even
} // find_median()
```

- This is **Theta(n log(n))**, bottlenecked by the sort. Both algorithms below aim for **linear time**.

15.4.1 Quickselect

- Finds the **k-th smallest element** in **average-case Theta(n)** time. Built on quicksort's **partitioning** step.
- After partitioning, one side of the pivot must contain the median (unless the pivot itself is the median). Only recurse into **that one side**.
- Algorithm, assuming the median should be at index k:
  1. Select a pivot.
  2. Partition into values less than the pivot and values greater than the pivot.
  3. Compare k to the pivot's index after partitioning:
     - k **less than** the pivot index: recurse on the **left**.

- k **greater than** the pivot index: recurse on the **right**.
- k **equal to** the pivot index: the value at the pivot is the median.

Example on [9, 4, 3, 5, 2, 8, 7, 1, 6]
```
select pivot 6, partition → [1, 4, 3, 5, 2, 6, 7, 9, 8]
median must be left of 6, recurse left, select pivot 2, partition → [1, 2, 3, 5, 4, 6, 7, 9, 8]
median must be right of 2, recurse right, select pivot 4, partition → [1, 2, 3, 4, 5, 6, 7, 9, 8]
median must be right of 4, recurse right. 5 is the median.
```

```
int32_t partition(std::vector<double>& vec, int32_t left, int32_t right) {
  int32_t pivot = --right;
  while (true) {
    while (vec[left] < vec[pivot]) {
      ++left;
    } // while
    while (left < right && vec[right - 1] >= vec[pivot]) {
      --right;
    } // while
    if (left >= right) {
      break;
    } // if
    std::swap(vec[left], vec[right - 1]);
  } // while
  std::swap(vec[left], vec[pivot]);
  return left;
} // partition()

double quickselect(std::vector<double>& vec, int32_t left, int32_t right, int32_t target_idx) {
  if (left + 1 >= right) {          // only one element in the range
    return vec[left];
  } // if
  int32_t pivot_idx = partition(vec, left, right);
  if (target_idx == pivot_idx) {    // found it
    return vec[target_idx];
  } // if
  if (target_idx < pivot_idx) {     // recurse left
    return quickselect(vec, left, pivot_idx, target_idx);
  } // if
  return quickselect(vec, pivot_idx + 1, right, target_idx);   // recurse right
} // quickselect()

double find_median(std::vector<double>& vec) {
  if (vec.empty()) {
    std::cerr << "Vector cannot be empty!" << std::endl;
    throw std::invalid_argument("Cannot find median of an empty vector");
  } // if
  if (vec.size() % 2 == 1) {        // odd
    return quickselect(vec, 0, vec.size(), vec.size() / 2);
  } // if
  return 0.5 * (quickselect(vec, 0, vec.size(), vec.size() / 2)      // even
              + quickselect(vec, 0, vec.size(), vec.size() / 2 - 1));
} // find_median()
```

Complexity

- **Average case Theta(n).** If each pivot splits roughly in half, and we only recurse into one side, the work per level halves:

```
T(n) = n + n/2 + n/4 + n/8 + n/16 + ... = 2n = Theta(n)
```

- This is an optimistic assumption, but as with quicksort's average-case analysis, the sum of a geometric series shows the total stays a **constant multiple of n** for any other constant-factor split.
- **Worst case Theta(n^2)** if you pick the worst pivot every time (the range shrinks by one instead of by half).
- In practice the worst case rarely happens, and quickselect behaves linearly for most real use cases.

15.4.2 Median of Medians

- Goal: guarantee **worst-case Theta(n)** by guaranteeing that each pivot splits the input into roughly equal halves.
- Randomization reduces the chance of the worst case (and is very effective in practice) but does not eliminate it. Median of medians eliminates it.

Algorithm

1. Divide the elements into **groups of size five**. The last group may have fewer than five; you may also drop it entirely, since this step only exists to pick a good pivot.
2. Find the **median of each subgroup** by sorting or brute force. Groups have at most five elements, so treat this as a **constant time** operation per group.
3. Store the subgroup medians in a **separate array**.
4. Use **quickselect** to find the median of that array, recursively using median of medians to choose its pivot.
5. Use the resulting pivot to **partition the original container**:
   - More than half the elements to the left: recurse left.
   - More than half the elements to the right: recurse right.
   - Pivot lands at the median's index: return its value (with extra logic for an even number of elements).

Why groups of five? An **odd** size makes median finding straightforward, and the size should be as small as possible so each subgroup median is cheap. Groups of **three** do **not** give worst-case Theta(n). Groups of **seven or more** do give Theta(n), but with a larger constant overhead since bigger subgroups cost more to process.

Example on 12 11 4 6 17 | 16 22 1 23 19 | 9 3 5 14 10 | 18 20 8 13 2 | 21 7 15
```
subgroup medians: 11, 19, 9, 13, 15
quickselect the median of [11, 19, 9, 13, 15] → 13
13 is used as the pivot for partitioning the original array.
```

Guarantee: at least 30 percent of values are discarded per partition

- Lay the subgroups out in order of their medians, each subgroup sorted vertically. Then:
  - Values to the **northwest** of the median of medians cannot be **larger** than it.
  - Values to the **southeast** of the median of medians cannot be **smaller** than it.

```
 3   4   2   7   1
 5   6   8  15  16
 9  11  13  21  19   ← row of subgroup medians, median of medians = 13
10  12  18   -  22
14  17  20   -  23
```

- With n values there are n/5 groups. For **half** of those groups, **three of five** values must lie in the northwest or southeast region.
- Count of values at least as extreme as the median of medians:

```
(1/2 * n) * (3/5) = 3n/10
```

- So at least **3/10 of all values** can be discarded after each partition, even in the worst case.

Footnote: groups with fewer than five elements can affect this result, but only by a constant amount, since there are at most five elements per group and only one group can

be short. The caveat does not change the result.

Recurrence and proof of Theta(n)

Work done at each iteration:

- Find the median of each group of five: constant per group, n/5 groups, so **n/5 * Theta(1) = Theta(n)**.
- Store the n/5 medians and recursively find their median: a subproblem of size **n/5**.
- Partition the original values with that pivot: **Theta(n)**.
- Recurse into the side holding the median. In the worst case that side has **70 percent** of the values: a subproblem of size **7n/10**.

```
T(n) = T(n/5) + T(7n/10) + n
```

- Recursion tree work per level: level 0 does n. Level 1 does n/5 + 7n/10 = 9n/10. Level 2 does n/25 + 7n/50 + 7n/50 + 49n/100 = 81n/100.
- So level k does $n * (9/10)^k$. The longer branch takes $\log_{10/7}(n)$ levels to hit the base case (the left branch terminates earlier, after $\log_5(n)$ levels), giving an upper bound:

```
T(n) = n + n(9/10) + n(9/10)^2 + n(9/10)^3 + ... + n(9/10)^(log_{10/7}(n))

     = n [ (9/10)^0 + (9/10)^1 + (9/10)^2 + ... + (9/10)^(log_{10/7}(n)) ]
```

- By the sum of an infinite geometric series, the bracketed term is a constant smaller than 10:

```
sum from i = 0 to infinity of (9/10)^i  =  1 / (1 - 9/10)  =  10
```

- Therefore **T(n) is bounded by Theta(n)**. Since the analysis assumed the worst possible pivot every time, the **worst-case** complexity of median finding with median of medians is **Theta(n)**.

Practical note: even though median of medians is asymptotically optimal in the worst case, **picking a pivot randomly is almost always sufficient in practice**. Computing the median of medians is not trivial, and the chance of hitting a worst-case pivot every time is very low even with random selection. For most use cases the overhead is not a worthwhile tradeoff.